

Week 3 - Lecture 6 - Problem: Generators and Safe File Reading

ILO

- **KU2:** Choose appropriate data types and data structures for different kinds of data.
- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, and generators.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS9:** Use standard libraries and follow good programming practices.

Objectives

After completing this problem, you should be able to:

1. Explain the difference between `return` and `yield`.
2. Write a simple generator function.
3. Read a text file line by line.
4. Remove whitespace and skip empty lines.
5. Catch `ValueError` and handle invalid data safely.
6. Build a dictionary from file content.

Background

A normal function uses `return` to finish and send back one result. A generator uses `yield` to send back values one at a time. This is useful when we read a file and want to process each line without storing every line in a list first.

In this problem, we will read a text file containing contact information. Each line contains a contact in the form:

```
name;phone_number
```

Example:

```
Alice; [070-123 45 67]  
Bob; [073-345 67 89]  
Carol; [076-456 78 90]
```

Some lines may be broken or empty. The program should skip invalid lines instead of crashing.

Task

Part 1: Compare `return` and `yield`

Write two small functions:

```
def numbers_return():  
    return [1, 2, 3]  
  
def numbers_yield():  
    yield 1  
    yield 2  
    yield 3
```

Then test both functions:

```
print(numbers_return())  
for value in numbers_yield():  
    print(value)
```

Explain the difference between the two functions in your own words.

Part 2: Write a simple generator

Create a function `my_range(start, stop, step=1)` that yields integers from `start` up to, but not including, `stop`.

Example:

```
for number in my_range(1, 10, 2):  
    print(number)
```

Expected output:

```
1  
3  
5  
7  
9
```

Your function must use `yield`.

Part 3: Parse one contact line

Write a function `parse_contact(line)` that receives one line of text.

The function should:

1. Remove surrounding whitespace and any unwanted characters.
2. Skip empty lines.
3. Split the line using `;`.
4. Return a tuple `(name, phone)` for a valid line.
5. Return `None` for an invalid line.

Example:

```
parse_contact("Alice;070-123 45 67")
# ("Alice", "070-123 45 67")

parse_contact("Broken line")
# None
```

Use `try` and `except` so that a line with too few or too many values does not crash the program.

Part 4: Read the file one line at a time

Write a generator function `contact_generator(filename)` that reads a file and yields valid contact tuples.

```
def contact_generator(filename):
    with open(filename, "r", encoding="utf-8") as file:
        for line in file:
            contact = parse_contact(line)
            if contact is not None:
                yield contact
```

Then write a function `load_contacts(filename)` that loops through the generator and stores the names as keys and phone numbers as values in a dictionary.

Example:

```
contacts = load_contacts("contacts.txt")
print(contacts)
```

Part 5: Handle invalid data safely

Write a short example showing how `ValueError` can happen.

```
try:
    number = int("abc")
except ValueError:
    print("That is not a valid integer.")
```

Then explain in one or two sentences why `try` and `except` are useful when reading files.

Example Run

```
for contact in contact_generator("contacts.txt"):
    print(contact)
```

Possible output:

```
('Alice', '070-123 45 67')
('Bob', '073-345 67 89')
('Carol', '076-456 78 90')
```

Requirements

- Use `yield` in at least one generator function.
- Explain the difference between `return` and `yield`.
- Read a file line by line instead of loading the whole file at once.
- Strip whitespace from each line.
- Skip empty lines.
- Handle invalid lines without crashing.
- Use `try` and `except` for safe error handling.
- Build a dictionary from valid contact entries.
- Keep the solution simple and beginner-friendly.

Discussion Questions

1. What is the difference between `return` and `yield`?
2. When does a generator function start running?
3. Why is it useful to read a file one line at a time?
4. What happens if `line.split(";")` gives the wrong number of values?
5. Why should invalid lines be skipped instead of stopping the program?
6. Why do we use `strip()` when reading text from a file?

Extension Challenges

1. Make the contact file contain a blank line and handle it safely.
2. Allow names or phone numbers to have extra spaces before or after the semicolon.
3. Add a second dictionary entry for a contact already in the file and print a message instead of overwriting it.
4. Write a generator that reads only the first 5 valid contacts.
5. Create a second file with bad data and test that your program still works.
6. Write valid readings to a new output file while the input file is processed.